

RAG Legal Assistant

Production Deployment Guide

FastAPI

ChromaDB

Groq LLM

SQLite

Python 3.10+

Uvicorn

Project: **rag-legal-assistant-master**

Date: June 2026 | Author: BaoNHN

Table of Contents

- 1. System Architecture Overview
- 2. Server Requirements
- 3. Security Hardening (Critical)
- 4. Installation on Ubuntu Server
- 5. Database & Vector Store Setup
- 6. Run as a System Service (systemd)
- 7. Nginx Reverse Proxy + HTTPS
- 8. Docker Deployment (Alternative)
- 9. Groq API Key Management
- 10. Quick Demo with Ngrok
- 11. Production Checklist
- 12. Troubleshooting

1. System Architecture Overview

This is a Vietnamese legal Q&A chatbot built on a Retrieval-Augmented Generation (RAG) pipeline. The stack:

Component	Technology	Purpose
Web Framework	FastAPI + Uvicorn	HTTP API & HTML interface
LLM	Groq API (llama-3.1-8b-instant)	Answer generation
Embeddings	BAAI/bge-small-en-v1.5 (HuggingFace)	Semantic search vectors
Vector DB	ChromaDB (local, persisted)	Law document retrieval
Chat DB	SQLite (chat.db)	Users, sessions, history
OCR	VietOCR + pdf2image + Poppler	PDF law document ingestion
Session	Starlette SessionMiddleware	Login state (cookie-based)

Architecture Flow

User → Nginx (HTTPS) → Uvicorn (FastAPI) → RAG Engine → ChromaDB + Groq API → Response

2. Server Requirements

2.1 Minimum Hardware

Resource	Minimum	Recommended
CPU	2 cores	4 cores
RAM	4 GB	8 GB (embedding model needs ~1–2 GB)
Disk	20 GB	50 GB (ChromaDB + model cache)
GPU	Not required	Optional – speeds up OCR with VietOCR

2.2 Software Prerequisites

- Ubuntu 22.04 LTS (or 20.04)
- Python 3.10 or 3.11
- pip 23+
- Nginx (for reverse proxy)
- Poppler utilities (for pdf2image)
- Certbot (for free HTTPS via Let's Encrypt)
- Internet access (to download HuggingFace model on first run)

3. Security Hardening (Critical — Do Before Deploying)

⚠ These vulnerabilities **MUST** be fixed before going live:

The current code has several insecure defaults that are only safe for local development.

3.1 Change the Session Secret Key

In `app.py` line 20, the secret key is hardcoded as `"secret_key"`. Replace it with a strong random value:

```
# Generate a secure key (run once in terminal):
python -c "import secrets; print(secrets.token_hex(32))"

# In app.py, replace:
app.add_middleware(SessionMiddleware, secret_key="secret_key", max_age=7200)

# With (use your generated value):
app.add_middleware(SessionMiddleware, secret_key=os.environ["SESSION_SECRET"], max_age=7200)
```

3.2 Move Groq API Key to Environment Variable

The file `groqkey.txt` should not be on disk in production. Move it to an environment variable:

```
# In engine/rag_engine.py, replace the file-read block:
# with open(..., "groqkey.txt") as f:
#     GROQ_API_KEY = f.read().strip()

# With:
GROQ_API_KEY = os.environ["GROQ_API_KEY"]
```

Then set it in your systemd service file or `.env` (see Section 6).

3.3 Hash Passwords in the Database

Passwords are currently stored in plaintext in SQLite. For production, use bcrypt:

```
pip install bcrypt

# In database.py – hash on user creation:
import bcrypt
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

# On login verification:
bcrypt.checkpw(password.encode(), stored_hash.encode())
```

3.4 Change Default Passwords

The seed accounts in `database.py` have weak passwords. Update them immediately after first run:

Account	Default Password	Action
testStudent1	123456P@ss	Change or delete
teacher1	Teacher@123	Change immediately

3.5 Restrict File Permissions

```
# On the server, restrict sensitive files:  
chmod 600 groqkey.txt  
chmod 600 chat.db  
chmod 700 chroma_db/
```

4. Installation on Ubuntu Server

1

Update system & install dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3.11 python3.11-venv python3-pip \
    git nginx poppler-utils \
    libgl2.0-0 libsm6 libxrender1 libxext6
```

2

Clone the project

```
cd /opt
sudo git clone <your-repo-url> rag-legal-assistant
sudo chown -R $USER:$USER /opt/rag-legal-assistant
cd /opt/rag-legal-assistant
```

3

Create virtual environment & install Python packages

```
python3.11 -m venv venv
source venv/bin/activate

# Install CPU-only PyTorch first (no GPU on most VPS):
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

# Install all other dependencies:
pip install -r requirements.txt
```

4

Set environment variables

```
# Create /opt/rag-legal-assistant/.env (never commit this file!)
cat > .env <<'EOF'
GROQ_API_KEY=your_groq_api_key_here
SESSION_SECRET=your_generated_secret_here
EOF

chmod 600 .env
```

5

Build the ChromaDB vector store

```
# Upload your law documents first, then:
source venv/bin/activate

# From dataset (Excel):
```

```
python database/build_db_from_dataset.py

# OR from text files:
python database/build_db_from_txt.py

# OR from PDF files:
python database/build_db_from_pdf.py
```

Note:

The HuggingFace embedding model (BAAI/bge-small-en-v1.5, ~130 MB) will be downloaded automatically on first run. Ensure internet access.

6**Test the app manually**

```
source venv/bin/activate
export GROQ_API_KEY="your_key"
export SESSION_SECRET="your_secret"
uvicorn app:app --host 127.0.0.1 --port 8000
# Open http://127.0.0.1:8000 to verify it works
```

5. Database & Vector Store Setup

5.1 File Locations

File/Folder	Purpose	Notes
chat.db	SQLite – users, chats, messages	Auto-created on startup
chroma_db/	ChromaDB vector store	Build once with build_db_*.py
uploads_tmp/	Temporary PDF uploads	Auto-created; safe to clear
groqkey.txt	Groq API key (legacy)	Replace with env variable

5.2 Backup Strategy

```
# Backup SQLite database daily (cron):
0 2 * * * cp /opt/rag-legal-assistant/chat.db /backups/chat_$(date +%Y%m%d).db

# Backup ChromaDB vector store weekly:
0 3 * * 0 tar -czf /backups/chroma_$(date +%Y%m%d).tar.gz /opt/rag-legal-assistant/chroma_db/
```

6. Run as a System Service (systemd)

This keeps the app running in the background, auto-restarts on crash, and starts on server reboot.

1

Create the service file

```
sudo nano /etc/systemd/system/rag-legal.service
```

```
[Unit]
Description=RAG Legal Assistant (FastAPI)
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/rag-legal-assistant
EnvironmentFile=/opt/rag-legal-assistant/.env
ExecStart=/opt/rag-legal-assistant/venv/bin/uvicorn app:app \
    --host 127.0.0.1 \
    --port 8000 \
    --workers 2 \
    --log-level info
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

2

Enable & start the service

```
sudo systemctl daemon-reload
sudo systemctl enable rag-legal
sudo systemctl start rag-legal

# Check status:
sudo systemctl status rag-legal

# View logs:
sudo journalctl -u rag-legal -f
```

Workers note:

ChromaDB and SQLite are not safe to share across multiple processes. Keep `--workers 1` unless you migrate to PostgreSQL + a networked vector store (Qdrant/Weaviate).

7. Nginx Reverse Proxy + HTTPS

1

Create Nginx site config

```
sudo nano /etc/nginx/sites-available/rag-legal
```

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    # Redirect all HTTP → HTTPS
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com www.yourdomain.com;

    # SSL (managed by Certbot – see next step)
    ssl_certificate      /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/yourdomain.com/privkey.pem;
    include              /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam          /etc/letsencrypt/ssl-dhparams.pem;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";

    # Increase upload limit for PDF files
    client_max_body_size 50M;

    # Proxy to FastAPI
    location / {
        proxy_pass          http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection "upgrade";
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }

    # Serve static files directly (better performance)
    location /static/ {
        alias /opt/rag-legal-assistant/static/;
        expires 7d;
    }
}
```

```
}  
}
```

2

Enable site & get SSL certificate

```
sudo ln -s /etc/nginx/sites-available/rag-legal /etc/nginx/sites-enabled/  
sudo nginx -t    # test config  
sudo systemctl reload nginx  
  
# Install Certbot and get free SSL:  
sudo apt install certbot python3-certbot-nginx -y  
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

3

Auto-renew SSL certificate

```
# Certbot adds this automatically, verify with:  
sudo systemctl status certbot.timer  
  
# Test renewal manually:  
sudo certbot renew --dry-run
```

8. Docker Deployment (Alternative)

If you prefer containerized deployment, create the following files in the project root:

8.1 Dockerfile

```
FROM python:3.11-slim

# Install system dependencies (poppler for pdf2image, libGL for OpenCV)
RUN apt-get update && apt-get install -y \
    poppler-utils libgl2.0-0 libsm6 libxrender1 libxext6 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Install CPU PyTorch first (keeps image smaller than CUDA version)
RUN pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Pre-download the embedding model so container doesn't need internet at runtime
RUN python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('BAAI/bge-small-en-v1.5')"

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1"]
```

8.2 docker-compose.yml

```
version: "3.9"
services:
  rag-legal:
    build: .
    ports:
      - "8000:8000"
    environment:
      - GROQ_API_KEY=${GROQ_API_KEY}
      - SESSION_SECRET=${SESSION_SECRET}
    volumes:
      # Persist database and vector store outside the container
      - ./chat.db:/app/chat.db
      - ./chroma_db:/app/chroma_db
      - ./uploads_tmp:/app/uploads_tmp
    restart: unless-stopped
```

8.3 Build & Run

```
# Create .env file with your secrets:
echo "GROQ_API_KEY=your_key_here" > .env
echo "SESSION_SECRET=$(python3 -c 'import secrets; print(secrets.token_hex(32))')" >> .env

# Build image (first time, takes 5-10 minutes):
docker compose build

# Start:
docker compose up -d

# View logs:
docker compose logs -f

# Stop:
docker compose down
```

Tip:

With Docker you still need Nginx in front for HTTPS. Run Nginx on the host and proxy to `localhost:8000`, or use a Nginx container with `docker compose`.

9. Groq API Key Management

Topic	Details
Get a key	Sign up at console.groq.com → API Keys → Create key
Free tier limits	~14,400 req/day, 6,000 tokens/min for llama-3.1-8b-instant
Rate limit handling	Already implemented in <code>_llm_invoke_with_retry()</code> – retries 3×
Model used	<code>llama-3.1-8b-instant</code> – fast, cheap, suitable for Vietnamese legal Q&A
Cost at scale	~\$0.05 per 1M input tokens, ~\$0.08 per 1M output tokens (as of 2025)

Never commit your Groq API key to Git.

Add `groqkey.txt` and `.env` to `.gitignore`.

```
# .gitignore - add these lines:
groqkey.txt
.env
chat.db
chroma_db/
uploads_tmp/
__pycache__/
```

```
venv/  
*.pyc
```

10. Quick Demo with Ngrok (Not for Production)

The project already has `start_ngrok.py` for exposing the local server to the internet for demos. Use this only for testing — not production.

```
# Install ngrok Python package:  
pip install ngrok  
  
# Make sure the app is running on port 8000 first:  
uvicorn app:app --host 127.0.0.1 --port 8000  
  
# In another terminal, start the tunnel:  
python start_ngrok.py  
# → Prints a public URL like: https://xxxx.ngrok-free.app
```

⚠ Ngrok limitations:

The token in `start_ngrok.py` is hardcoded and publicly visible in your repo. Rotate it at **dashboard.ngrok.com** immediately. Free tier URLs change on every restart. Do not use for real users.

11. Production Checklist

Category	Item	Status
Security	Session secret moved to environment variable	☐
	Groq API key moved to environment variable	☐
	Default passwords changed (testStudent1, teacher1)	☐
	groqkey.txt and .env added to .gitignore	☐
Infrastructure	Nginx configured with HTTPS (Let's Encrypt)	☐
	systemd service created and enabled	☐
	Firewall configured (only ports 80, 443, 22)	☐
Data	ChromaDB vector store built from law documents	☐
	Database backup cron job configured	☐
	uploads_tmp/ directory write-permission set	☐
Testing	Login as student and ask a legal question – works?	☐
	Login as teacher and import a PDF – works?	☐
Monitoring	journalctl / logs accessible	☐
	Groq API rate limit alerts set up (optional)	☐

12. Troubleshooting

Problem	Likely Cause	Fix
<code>ModuleNotFoundError: vietocr</code>	vietocr requires torch to be installed first	Run <code>pip install torch</code> before <code>pip install -r requirements.txt</code>
<code>pdf2image.exceptions.PDFInfoNotInstalledError</code>	Poppler not installed on Linux	<code>sudo apt install poppler-utils</code>
<code>chroma_db not found</code>	Vector store was not built yet	Run <code>python database/build_db_from_dataset.py</code>
Groq <code>429 Too Many Requests</code>	Rate limit exceeded	The app retries automatically; upgrade Groq plan for higher limits

Session lost on restart	Session secret changes between restarts	Set a fixed <code>SESSION_SECRET</code> env variable (not random each time)
App slow to start	HuggingFace model downloading (first run)	Pre-download model with: <code>python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('BAAI/bge-small-en-v1.5')"</code>
Nginx 502 Bad Gateway	FastAPI not running or wrong port	Check <code>sudo systemctl status rag-legal</code> and that port 8000 is listening
File upload fails (>1MB)	Nginx default body size limit	Add <code>client_max_body_size 50M;</code> to Nginx config

Useful Commands

```
# View live app logs:
sudo journalctl -u rag-legal -f

# Restart the app:
sudo systemctl restart rag-legal

# Test if app is running on port 8000:
curl http://127.0.0.1:8000/

# Check Nginx errors:
sudo tail -f /var/log/nginx/error.log

# Check open ports:
ss -tlnp | grep 8000
```